Understanding and Optimizing Multi-Stage AI Inference Pipelines

Abhimanyu Rajeshkumar Bambhaniya[1], Hanjiang Wu[1], Suvinay Subramanian[2], Sudarshan Srinivasan[3], Souvik Kundu[4], Amir Yazdanbakhsh[5], Midhilesh Elavazhagan[3], Madhu Kumar[3], Tushar Krishna[1]
[1]Georgia Institute of Technology, [2]Google, [3]Intel, [4]Intel Labs [5]Google DeepMind,
Corresponding email: abambhaniya3@gatech.edu

arXiv:2504.09775v2 [cs.AR] 16 Apr 2025

***Abstract*—The rapid evolution of Large Language Models (LLMs) has driven the need for increasingly sophisticated inference pipelines and hardware platforms. Modern LLM serving extends beyond traditional prefill-decode workflows, incorporating multi-stage processes such as Retrieval-Augmented Generation (RAG), key-value (KV) cache retrieval, dynamic model routing, and multi-step reasoning. These stages exhibit diverse computational demands, requiring distributed systems that integrate GPUs, ASICs, CPUs, and memory-centric architectures. However, existing simulators lack the fidelity to model these heterogeneous, multi-engine workflows, limiting their ability to inform architectural decisions.**

To address this gap, we introduce HERMES, Heterogeneous Multi-stage LLM inference Execution Simulator. HERMES models diverse request stages—including RAG, KV retrieval, reasoning, prefill, and decode—across complex hardware hierarchies. Unlike prior frameworks, HERMES supports heterogeneous clients executing multiple models concurrently while incorporating advanced batching strategies and multi-level memory hierarchies. By integrating real hardware traces with analytical modeling, HERMES captures critical trade-offs such as memory bandwidth contention, inter-cluster communication latency, and batching efficiency in hybrid CPU-accelerator deployments. Through case studies, we explore the impact of reasoning stages on end-to-end latency, optimal batching strategies for hybrid pipelines, and the architectural implications of remote KV cache retrieval. HERMES empowers system designers to navigate the evolving landscape of LLM inference, providing actionable insights into optimizing hardware-software co-design for next-generation AI workloads.

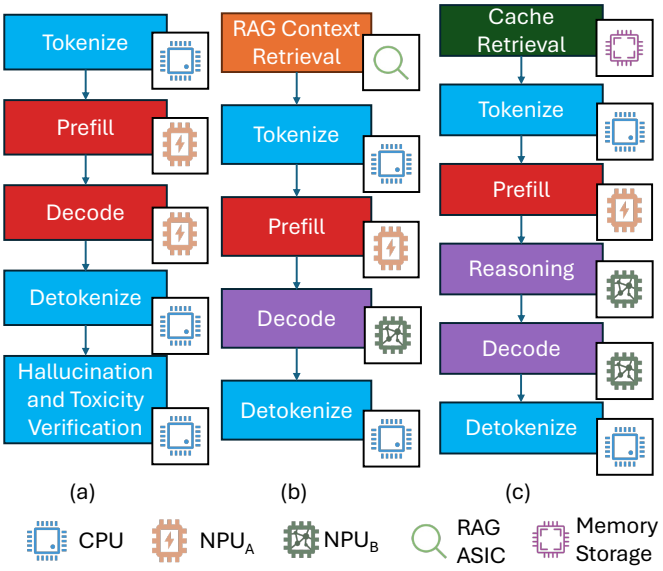


Fig. 1: Three example LLM Inference requests executed on different clients. a.) Basic LLM inference pipeline followed by Hallucination & safeguards verifications, b.) RAG-based LLM inference pipeline with disaggregated serving, c.) Past memory retrieval LLM inference pipeline with reasoning.

I. Introduction

Large Language Models (LLMs) have emerged as transformative tools across a vast spectrum of applications, from real-time conversational agents and code generation to scientific domains like protein sequencing [1], chemical property prediction [2], and video synthesis [3]. Commercial systems such as ChatGPT, Gemini, and GitHub Copilot [4], [5], [6] exemplify their versatility, often surpassing human performance in specialized tasks [7]. The scaling laws governing LLMs [8] suggest that larger models, now exceeding 1.8 trillion parameters [9], demand increasingly sophisticated hardware platforms. These platforms must integrate heterogeneous components—GPUs, ASICs, memory-centric nodes, and CPUs—into distributed systems capable of balancing compute, memory, and interconnect resources [10], [11], [12].

Modern LLM inference pipelines, however, extend far beyond the traditional prefill and decode stages. Real-world deployments now incorporate multi-stage workflows, including Retrieval-Augmented Generation (RAG), key-value (KV) cache retrieval, dynamic model routing, multi-step reasoning, and post-processing. For instance, RAG requires tight coupling between retrieval engines and inference accelerators, while reasoning stages may demand iterative CPU-accelerator interactions. These stages exhibit divergent computational profiles: RAG stresses memory bandwidth, reasoning relies on low-latency interconnects, and decode stages prioritize high-throughput token generation. Yet, existing simulators [13], [14] remain confined to static prefill-decode pipelines, ignoring the interplay between heterogeneous hardware and multi-engine workflows. This limitation obscures critical architectural trade-offs, such as the impact of memory hierarchy design on past context retrieval latency or the optimal batching strategy for hybrid CPU-accelerator systems.

The hardware landscape further complicates this challenge. Next-generation platforms must orchestrate diverse

Framework	LLM Serving Pipeline Modeling					
	Coordinator Modeling		Engine Modeling		Cluster Modeling	
	Coordinator Clients	Models supported	Request Stages	Batching Type	NPU & Network	Memory Hierarchy
Vidur [14]	Multiple homogeneous Client	Single model	Prefill, Decode	Static, Continuous, Chunked	Real HW data + ML prediction	Single-Level
LLMServingSim [13]	Multiple NPUs + CIMs Client	Single model	Prefill, Decode	Continuous	Roofline from astra-sim	Single-Level + Offload)
Splitwise-sim [15]	Three heterogeneous pools†	Single model	Prefill, Decode	Global Disaggregated	Real HW data + ML prediction	Single-Level
HERMES (ours)	**Multiple heterogeneous Client**	**Multiple simultaneous models**	**Cache Retrieval**, **RAG**, Prefill, **Reasoning**, Decode	Static, Continous, Chunked, **Global/Local Disaggregated**	Real HW data or external simulators + ML prediction	**Multi-Level** + Offload)

TABLE I: Comparison of prior works for modeling LLM inference against this work.† Splitwise-sim instantiates prefill, decode and mixed pools with all clients within a pool having same hardware. HERMES allows arbitrary heterogeneous groupings.

clients—GPUs for tensor parallelism, ASICs for memory-bound tasks like KV retrieval, CPUs for reasoning, and CXL-attached memory pools for offloading—across hierarchical clusters (Fig. 1). Compounding this, emerging batching techniques such as chunked prefill [16], and disaggregated prefill-decode [17] introduce additional variables into the design space. We define a single hardware *client* as hardware cluster combined with scheduler for request scheduling. The hardware cluster includes hardware, memory, and other physical components combined with software optimization technique specific to a particular hardware. For, e.g. 2xH100 GPU running vLLM is considered a single client, a separate ASIC device for running RAG steps would be another client.

Current simulation frameworks, Table I, lack the fidelity to model these dynamics. For example, while Vidur [14] supports multiple homogeneous clients(for existing system only), it cannot simulate heterogeneous client configurations or disaggregated prefill and decode hardware, particularly for systems that are not yet accessible. Similarly, LLMServingSim [13] fail to capture both chunked and disaggregated batching. Both these works falls short at modeling advanced multi-stage requests, rendering them inadequate for evaluating modern inference pipelines.

To bridge this gap, we introduce HERMES, a high-fidelity simulator designed for multi-stage, heterogeneous LLM serving systems. HERMES uniquely models diverse request stages like KV cache retrieval, RAG, reasoning, prefill, and decode—across a hierarchy of hardware components, including accelerators, memory nodes, and disaggregated clusters. Unlike prior works, HERMES supports multiple heterogeneous clients servicing distinct models and request types simultaneously, while integrating advanced batching strategies (e.g., chunked prefill and disaggregated batching) and multi-level memory hierarchies with offload capabilities. Furthermore, HERMES uses hardware traces with linear regression to model real hardware systems. At the same time, HERMES leverages open-source hardware modeling frameworks to simulate future systems, and provide recommendation for future hardware inference system design.

HERMES empowers computer architects to answer pivotal questions like: How does adding a parallel reasoning thoughts affect end-to-end latency under memory constraints? What is the optimal balance between chunked, continuous, and disaggregated batching for hybrid RAG-decode pipelines? Through various case studies, we demonstrate HERMES's various capabilities.

To this end, we make the following contributions:

- **A Heterogeneous Multi-Stage Simulation Framework**: HERMES is the first simulator to model distributed LLM serving pipelines with heterogeneous clients, multi-stage workflows, and multi-level memory hierarchies, bridging the gap between idealized models and real-world deployments.
- **Batching strategy for Frontier Multi-Stage LLM Inference** ⚙: With support for LLM stages like KV cache retrieval, Retrieval Augmented Generation and Reasoning throughput compute time scaling, HERMES is able to provide insights into optimal batching strategy of different pipelines.**Our key observations are**: i) *Disaggregated batching* is able provide highest *throughput/energy* in most cases. ii) *Chunked batching* provides *high throughput* and is able to *sustain higher request injection rate* but requires *relaxed TTFT SLOs*. iii) *Continuous batching* is *optimal for TTFT* is most cases, but is unable to sustain high injection rates. Table III summarizes the optimal batching strategy for different usecase, serving system size and varying optimization metrics.
- **Actionable Design Insights** 📟: Through case studies, we quantify critical trade-offs, such as i) interplay between RAG embedding model and retrieval components placements, ii) memory hierarchy design for remote KV cache retrieval on end-to-end pipeline efficiency.

By enabling precise exploration of these dimensions, HERMES provides a critical tool for architects navigating the complex interplay between evolving LLM workflows and future hardware platforms.

This paper is organized as follows. Section II introduces key concepts related to LLM inference pipeline stages and scheduling strategies.

Section III presents the design of HERMES , our modular simulation framework. We describe its architecture, hardware abstraction layers, ML-assisted runtime prediction, and integration with external simulators. In Section IV, we demonstrate how HERMES enables detailed exploration of modern software paradigms such as Retrieval-Augmented Generation (RAG) and scaling inference compute time with reasoning trees.

Section V highlights the practical utility of HERMES through case studies. First, in Section V-A, we use HERMES to unravel

the complex design space of batching strategies across different LLM pipelines. Then, in Section V-B, we investigate the impact of remote KV cache retrieval, quantifying how the physical placement of cache storage affects end-to-end latency in large-scale systems.

II. Background

A. LLM Inference Pipeline Stages

Most previous works primarily focus on analyzing inference performance based solely on the prefill and decode stages [14], [13]. However, their analyses may not fully capture the complexities of modern inference. Fig. 1 showcases the components within the workflow of modern inference that is covered in HERMES . The major listed components include *Prefill*, *Decode*, *RAG* and *Reasoning*.

Prefill: In the prefill phase, it does a single forward pass with the input prompt and generates the first output token. This stage is computationally intensive as it involves processing all input tokens at once.

Decode: Directly after Prefill, output tokens are generated sequentially in an autoregressive manner. Each newly generated token is fed back into the model to produce the next token until end-of-sequence token is generated. To process a single token, it is mostly memory-bounded and will be more performant if multiple decode tokens are batched together.

Reasoning breaks down a complex task into multiple smaller steps, reasoning-based models enable LLMs to generate more accurate answers for problems that require critical thinking and a structured thought process. In real-world applications, this approach typically necessitates multiple rounds of forward passes through the model, with each intermediate step refining the reasoning by building upon previous outputs. Reasoning significantly increases computational load and memory requirements, leading to higher latency.

B. Serving Methods

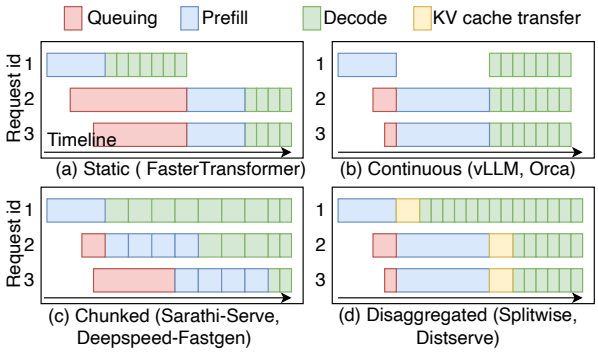


Fig. 2: Batching mechanisms and their latency impact on the prefill and decode phases.

Continuous Batching Continuous batching, widely used in modern inference systems, boosts compute throughput by prioritizing prefill and batching decode stages to enhance throughput [18], [19]. As shown in Fig. 2(b), Requests 2 and 3 preempt Request 1's decode, and all three decode together after their prefill, improving throughput over static batching (Fig. 2(a)).

Chunked Batching Chunked batching improves latency-throughput balance by splitting long sequences into smaller, fixed-size chunks. As shown in Fig. 2(c), this allows prefill (e.g., Request 2) to run alongside decode (e.g., Request 1), avoiding the stalls seen in prefill-prioritized strategies like continuous batching.

Disaggregated Batching Disaggregated serving decouples prefill and decode stages by assigning them to independently scaled hardware instances, enabling flexible resource allocation for heterogeneous workloads. For example in Fig. 2(d), Request 1 begins decoding while Requests 2 and 3 are still in prefill, due to this separation. Decode stages are then batched as resources become available, improving throughput. But it comes with the cost of KV cache transfer.

We define two disaggregation types: *Global*, as in Splitwise [17], uses a shared GPU pool without locality constraints, offering better load balancing; and *Local*, which restricts requests to fixed, physically co-located GPU groups, reducing KV cache transfer overhead. By default, we use global disaggregated batching unless otherwise noted.

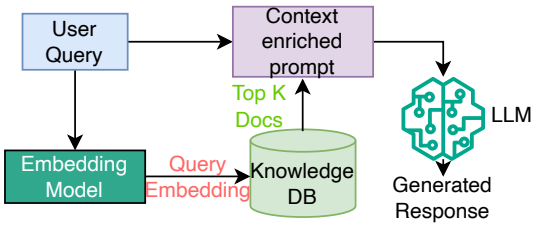


Fig. 3: RAG pipeline

C. Retrieval Augmented Generation

Retrieval-Augmented Generation (RAG) enhances LLM inference by incorporating external knowledge to improve factual accuracy and contextual relevance. As illustrated in Fig. 3, the process consists of two main stages: (1) retrieving relevant documents through embedding model from a vector database using approximate nearest neighbor (ANN) methods such as FAISS [20] , and (2) generating responses using the LLM with the augmented prompt. This enhanced pipeline enhances the answer quality through combining model's internal knowledge with the retrieved external information [21].

We employ Dense Passage Retrieval (DPR) [22] to embed both queries and documents into high-dimensional vector spaces, allowing semantic similarity search. Among ANN techniques, Hierarchical Navigable Small World (HNSW) graphs enable fast retrieval through graph-based traversal but at high memory cost [23]. In contrast, Inverted File Index

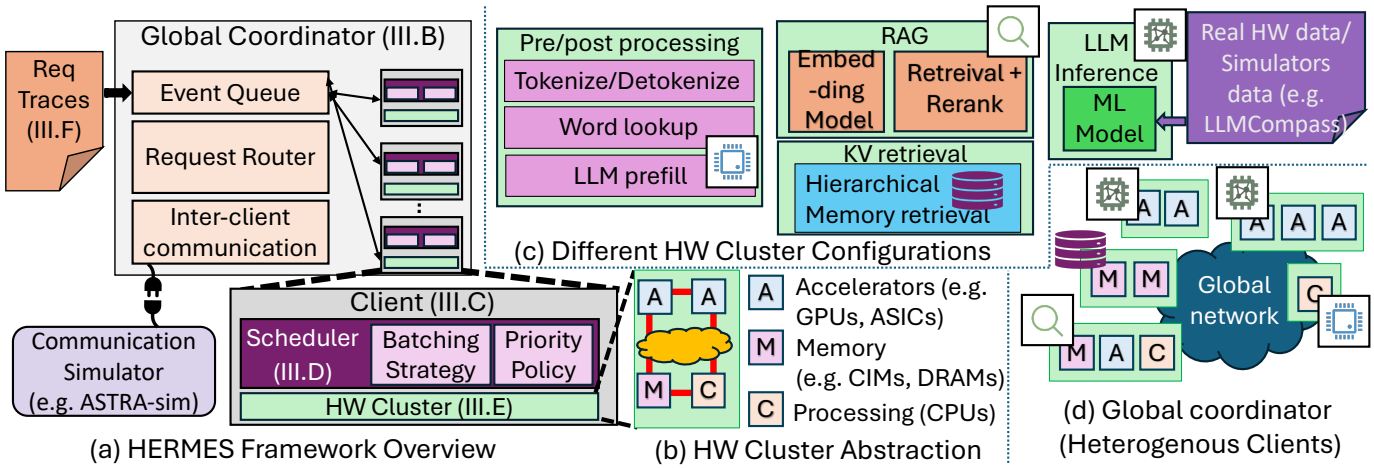


Fig. 4: HERMES overview. HERMES can simulate multiple heterogeneous clients simultaneously. Each clients consist of a engine (e.g. vLLM, Triton) which issues tasks to a hardware cluster(Nvidia HGX Box, Sambanova SN40L, Cerebras CS3, CPU host with offloading memory instance). The hardware cluster can be combination of NPUs, memory and CPUs.

(IVF) methods cluster the database into buckets and search only within a few, offering better memory efficiency. Product Quantization (PQ) is commonly used with IVF to compress the vector database with minimal recall degradation [24]. In this work, we use IVF-PQ as our default retrieval method to strike a balance between recall quality and memory footprint, which becomes critical at billion-scale vector datasets [25].

III. HERMES

We introduce HERMES(Fig. 4), a Heterogeneous Multi-stage LLM Inference Execution Simulator designed to capture the complexity of real-world LLM inference pipelines. Unlike traditional simulators that focus solely on prefill and decode stages, HERMES models the entire inference process—spanning pre/post processing, retrieval, caching, and generation; across heterogeneous hardware clusters.

HERMES simulates end-to-end execution of SOTA LLM inference pipelines over diverse hardware clients,Fig. 4(c), where each client is specialized for a subset of inference stages. These clients, Fig. 4(b) may represent combinations of GPUs, TPUs, CPUs, memory nodes, or ASICs, each configured to execute the specific stages of LLM inference pipeline. We use the term NPU to refer to these hardware components. The entire serving setup,Fig. 4(d), simulated by HERMES can have various heterogeneous clients responsible for different request stages and connected through a global network.

A. Overview and simulation methodology

HERMES is a high-fidelity discrete event simulator that models LLM inference as a sequence of composable stages. To flexibly model the diverse stages of the LLM inference pipeline, our simulation framework follows a hierarchical design: **Global Coordinator Section III-B** → **Client Section III-C** → **Scheduler Section III-D** → **Hardware Cluster Section III-E**. At the top level, the Global Coordinator receives input requests and routes them to the appropriate serving clients. Each Client is equipped with a dedicated scheduler that maps incoming requests onto the hardware cluster simulator. Finally, the HW Cluster simulator simulates the performance of the scheduled requests.

This modular design allows for the decoupling of software and hardware components, enabling seamless integration of either external hardware simulators or empirical runtime traces collected from real systems. Each request may consist of stages such as preprocessing, RAG, KV cache loading (user historical cache, or prefix cache or shared context cache), prefill, decode, and postprocessing. Each stage is modeled as a discrete event, and HERMES can simulate arbitrary mappings of these stages across heterogeneous HW clients.

Fig. 1 illustrates various request pipeline configurations and how their constituent stages are distributed among different clients. A global coordinator (see Section III-B) orchestrates the execution of these multi-stage pipelines by assigning and routing tasks across clients, Fig. 4(d). HERMES defines five client types, each supporting a distinct subset of inference stages, enabling rich and flexible simulation scenarios.

B. Global Coordinator

The global coordinator governs the end-to-end execution of inference requests across clients, ensuring that all stages are scheduled and executed in order, and that inter-stage communication is properly managed. It maintains a global event queue and handles two primary event types: • Request events: Triggered when a new request enters the system or when a client returns a request after completing a stage. • Client events: Represent the completion of a stage(batched or individual) by a client and the transfer of control back to the coordinator.

Additionally, we maintain a global clock to guarantee the sequential execution of events and engine step without any single client running faster than others.

1) Routing and Load Balancing: To determine the next client for a given request stage, the coordinator uses a routing module. When multiple clients are capable of executing the same stage, the router applies a configurable load-balancing policy to distribute work efficiently. Balancing requests across clients is critical to avoiding bottlenecks in multi-stage pipelines. We support three routing policies: Round Robin, Load-based, Heavy-Light split [26].

Load in the latter two policies can be defined using various request attributes, such as: i) input context length, ii) output context length, iii) current KV cache size, iv) tokens remaining to be generated. These metrics enable up to nine distinct routing strategies. HERMES has a highly modular router API allowing new routing policies to be integrated with minimal effort.

With multiple LLMs model instantiations, we assume each request contains metadata specifying the target model. [1] The router can also exploit global client placement information to minimize communication costs, especially in disaggregated settings where large KV caches must be transferred between clients.

2) Global Communication: Once a routing decision is made, the global communication simulator handles data transfers between clients. It estimates communication overhead based on data size and transfer granularity (e.g., full KV cache vs. layer-wise transfer [17]), depending on the transition between request stages. For simulating multi-level, heterogeneous interconnects, HERMES integrates with astra-sim [27], enabling accurate modeling of communication latency and bandwidth constraints. After data is transferred, the request is reinserted into the global event queue as a new request event, ready to be processed by the target client. algorithm 1 presents the core simulation loop used by the coordinator, integrating event scheduling, routing, and inter-client communication in a unified discrete event framework.

Algorithm 1: Global Coordinator Simulation Algorithm

```
Initialize engine connections
while request_serviced < request_accepted do
    Next Event Simulation
    if Event is Request-push then
        if Engine not allotted then
            Engine_next = Router(Request)
        end if
        Engine_next.add(Request)
        Activate engine if idle
    else if Event is Engine Step then
        Process engine step and collect completed requests
        Handle Request Completion
        for each completed request do
            if request processing is complete then
                Mark request as serviced
            else
                Engine_next = Router(Request)
                Start Engine transfer event.
                Enqueue request for next processing stage
            end if
        end for
    end if
end while
```

C. Clients

Each Client in HERMES is composed of a Scheduler and a Hardware Cluster model (details in Section III-D and Section III-E). We first describe the different types of Clients and then delve into the responsibilities of the Scheduler and Hardware Cluster models. Drawing inspiration from vLLM, each client operates at a step[2] granularity, with requests added asynchronously to the client. After HW cluster simulation completes the assigned request stage, client sends updated requests to the coordinator. We briefly discuss the roles of four type of clients in HERMES as shown in Fig. 4(c).

1) Preprocessing and Postprocessing Clients: The preprocessing clients are responsible for preparing input data before it is sent to the LLM inference stage. This includes operations such as tokenization, padding, truncation, and attention masking. These transformations are crucial for ensuring that the model receives inputs in the correct format and with consistent tensor dimensions.

Postprocessing clients, handle the final stage of the inference pipeline. Their responsibilities include detokenizing model outputs back into human-readable text and applying additional filtering mechanisms. For instance, many production systems include toxicity filters or bias detection modules, which can be implemented as simple rule-based lookups or small transformer-based classifiers. In reasoning based models, postprocessing may also involve reward model [28] inference—using either outcome-based (ORMs) or process-supervised (PRMs) models—to score generated responses.

2) RAG Clients: The RAG client is responsible for performing embedding generation and document retrieval prior to LLM inference. When a user query is received, it is first encoded into an embedding using a lightweight encoder model, such as NVIDIA's proprietary embedding models or the multilingual E5 series [29], [30], [31]. Once the embedding is obtained, a retrieval step follows, typically using an efficient indexing strategy such as IVF-PQ [32], [33], [34], to find the most relevant documents.

3) KV Cache Retrieval Clients: KV cache retrieval is a critical optimization for reducing time-to-first-token (TTFT) in modern inference systems [35], [36], [37]. A commonly used technique is Prefix Caching (PC) [38], where the KV cache of prior context tokens is reused when a new query shares a prefix token with an earlier query. This reuse can bypass the prefill computation, significantly reducing latency and compute load. In chat-based applications, these KV caches are often stored persistently across sessions to support continuity in multi-turn conversations.

[1]Future extensions may support adaptive model routing based on request characteristics such as complexity, quality, or priority.

[2]A step is defined as single inference pass

4) LLM inference clients: simulate the core stages—*prefill* and *decode*. LLM inference client can either run both prefill and decode stage on the same hardware cluster(used by continuous batching/chunked batching) or we can have separate clients for prefill and decode stage.

Our modular client definition allows users to define new clients with minimal effort.

D. Schedulers

Each client has a scheduler which assigns requests to executed at each step. We define two base scheduler: i) Batched: Used for single step tasks like word lookup. Batching all requests in the engine parallely will extract maximum reuse. ii) Sequential: For tasks without reuse possibility, e.g. padding and truncation, etc. We assign available cores to complete the tasks in linear fashion. Pre/post-processing client uses the sequential scheduler while RAG, and KV cache retrieval clients use the batched scheduler to maximize the efficiency.

1) LLM Scheduler: Since LLM inference requires multiple steps to complete the request, it requires a special scheduler. LLM scheduler enforces batching policies and is modeled after vLLM's scheduler. HERMES currently supports five batching strategies: *Static Batching* (FasterTransformers [39]) *Continuous Batching* (Orca/vLLM [18], [40]) *Chunked Batching* (Sarathi-Serve, DeepSpeed-FastGen [41], [42]) *Mixed Batching* (Splitwise Prefill [17]) *Disaggregated Batching* (Splitwise/Dist-Serve [17], [43]).

For each batching strategy, scheduler also supports flexible request packing policies such as *First-Come-First-Serve (FCFS)* and *Least Work Left*. The scheduler and batching API's are modular, allowing users to define custom packing or scheduling strategies with minimal effort.

In addition to batching policies, the scheduler enforces user-defined constraints such as the maximum number of batched tokens or batch size. Scheduler also manages on-device memory by preventing request admission when memory (e.g., KV cache) is insufficient and by evicting KV caches of completed requests.

E. Heterogeneous Cluster Modeling

1) ML-Assisted LLM Cluster Modeling: Previous works [14], [15] have used ML models to predict LLM forward pass runtime. Vidur uses random forest to predict runtime of each operator and aggregates the runtime of various operator to calculate a single forward pass time. Splitwise-sim use a piece-wise linear model for runtime and power estimations.

Similarly, we used real hardware data collecting over 58K datapoints on a DGX-H100 box running vLLM with LLaMA2-70B. We vary input size, batch size, chunk size (for chunked batching), and tensor parallelism (TP2/TP4/TP8). We observe that decode batches constitute ~96% of the dataset. We use polynomial regression models decode runtime with mean square error(MSE) = 4.09e-07. Prefill runtime is modeled using past token count, prefill token count, batch size, and token^2, with MSE = 6.49e-05. HERMES can also leverage analytical simulators *LLMCompass* [44], *GenZ* [45] to model unavailable system configuration(e.g. Nvidia Rubin [46] or Google Ironwood [47]). Additional framework like HW Simulators [27], [48], [49], [50] can be used to model individual operators/dataflow/microarchitectures. We can apply the same ML modeling approach to predict run times generated by analytical simulators.

While this approach models a particular model configuration, we can collect data points with different optimizations (such as quantization [51], [52], [53], pruning [54], [55], [56], [57], speculative decoding [58], [59], [60], [61], flash attention [62], [63]) and use it as to train the ML model. ML modeling provides a 20–50× simulation speedup compared to analytical simulation.

2) RAG cluster:: RAG HW cluster required to i) convert input query into search space embedding, ii) retrieve related documents, iii) Re-rank top k documents. In practice, the embedding and retrieval workloads can run on different devices. For instance, the embedding model may be deployed on NPUs or GPUs, while retrieval and reranking steps are more suited to CPUs.

For embedding model, we use the embedding model prefill time for a give query. The time is either calculated from real trace or simulator as described in previous subsection.

For modelling the retrieval and reranking steps are more suited to CPUs we implement IVF-PQ modelling equations described in RAGO-SERVE [34].

3) KV Retrieval:: HERMES models KV cache retrieval as a multi-level memory hierarchy, similar in spirit to CPU cache systems. Each level in the hierarchy is characterized by its capacity, lookup latency (ranging from nanoseconds to milliseconds), bandwidth, and cache hit rate. However, unlike CPU caches where a miss leads to DRAM access, a miss in prefix caching may result in the need to recompute the entire context using the LLM, which is significantly more expensive.

The expected retrieval latency for a cache retrieval request with cache size, is computed recursively using the following expression $T_{\text{retrieval}} = f(\text{Size}_{KV}, C_1)$, where

$$f(\text{KV}, C_n) = \text{Hit}_n \cdot \left(T_{\text{lookup}_n} + \frac{\text{Size}_{KV}}{\text{BW}_n} \right) + (1 - \text{Hit}_n) \cdot f(\text{Size}_{KV}, C_{n+1}) \tag{1}$$

For cache n in cache hierarchy,Hit_n refers to hit rate of cache, T_{lookup_n} /BW_n refers to the lookup latency and retrieval bandwidth of the cache. at This formulation captures the expected latency by recursively aggregating the time cost of each cache level based on its hit probability. HERMES exposes this functionality through a modular API that allows easy integration of more sophisticated memory models in the future, including caching policies or asynchronous prefetching mechanisms.

4) Pre/Post Processing:: For Pre/post processing clients, we assume varying latency for different tasks. We use a forward pass on small LLM model(~2B) for modeling toxicity filters or bias detection. To model search word lookup, we model runtime proportional to number of generated tokens.

Each hardware cluster in HERMES is modular by design and can be replaced with a high-fidelity simulation model. This modularity allows users to selectively refine the simulation

of specific clusters and analyze how localized improvements impact end-to-end LLM inference performance.

F. Request Modeling

Each request passes through a sequence of execution stages (Fig. 1) with distinct compute and memory demands. RAG is typically handled by large-memory CPUs or ASICs [33], cache retrieval benefits from fast memory, and prefill/decode stages are suited for compute-heavy NPUs like GPUs. We model requests using: (1) input datasets characterizing workload patterns, and (2) output metrics and processing traces reflecting system behavior.

1) Input Datasets and Workloads: Inference begins with feeding a dataset of requests into the system.

Request size: To model diverse prefill and decode token workloads, we use a combination of real and synthetic traces.*Real traces* from production services, such as Azure trace [64] (Conv and Code), capturing realistic input-output token distributions. *Synthetic traces* are generated based on observed characteristics in common workloads. They are modeled as normal distribution with user configurable mean and variance for input and output tokens.

Request injection is modeled using a range of models including uniform, normal, poisson, and bursty distributions. This approach better reflects real-world traffic patterns and enables more robust evaluation of system behavior under diverse operational scenarios.

Additionally each request has additional parameters depending on the associated stages (e.g. RAG request might have required rag algorithm parameters).

2) Output Metrics Collection: HERMES collects detailed metrics during simulation to analyze how requests are processed across the system. These metrics inform performance insights and guide system design decisions. We categorize the collected data as follows:

Individual Request Metrics: For every request, we record fine-grained statistics, including: associated stage metrics (engine assignment time, start time, end time), for prefill and decode, we also maintain each token metrics(scheduled time, hardware start time and hardware end time).

Scheduler-Level Metrics: These metrics track the request load queued and processed at each simulation step. This includes: Instantaneous and average queue length, variations in arrival volume scheduling rate, step-wise memory load, and finished requests.

Client-Level Metrics: Each client instance maintains operational statistics through its scheduler. Tracked metrics include: Load and queue size at specific timepoints, Request service rate over time, Estimated power consumption.

Global Metrics: To capture holistic system behavior, we log aggregate statistics such as, serviced requests information, latency breakdowns (mean, T50, T90, T99), and communication metrics.

These global insights enable comprehensive evaluation of system performance and comparative analysis across configurations and scheduling strategies.

Request Tracing and Visualization: All request-level execution details are encoded in JSON format, capturing each stage of processing. This format enables seamless integration with visualization tools, such as Chrome Tracing.

Together, the input datasets and output metrics constitute the foundation of our request modeling pipeline, enabling rigorous, end-to-end analysis of inference system behavior under a wide range of workload conditions.

G. HERMES Fidelity

In this section, we demonstrate HERMES 's fidelity on end-to-end runtime predictions across different serving strategies. For single client simulation we compare LLama3-70B model and with varying input size, number of request and chunk size against the vLLM scheduler with chunked batching generating 200 output tokens for all requests. Fig. 6 shows the HERMES achieves less error (¡ 2% average error) in across varying hardware cluster size and serving chunk size.

To validate the effectiveness of disaggregated client serving, we utilize real request traces collected from the Azure platform [64]. Splitwise [65] implements disaggregated serving atop vLLM for small-scale system evaluation. For large-scale scenarios, Splitwise introduces an open-sourced simulation framework, splitwise-sim which is validated against their real-system implementation [15]. In our validation, we compare against their simulation framework as proxy for real-system simulation, as we lack access to large systems.

We simulate two different models(Llama-2-70B and Bloom-176B) on 80-GPU system configuration with 8 prefill clients and 2 decode clients under different request distributions (RPS=20 and RPS=40). Across usecases we observe minor different in modeling (¡ 6% maximum error) as shown in Fig. 5. This major difference in runtime arises from communication because splitwise-sim employs a dummy link-based communication model with specified lower-bound bandwidth number. In contrast, we use Astra-sim to model client communication, which introduces slight differences in overall runtime.

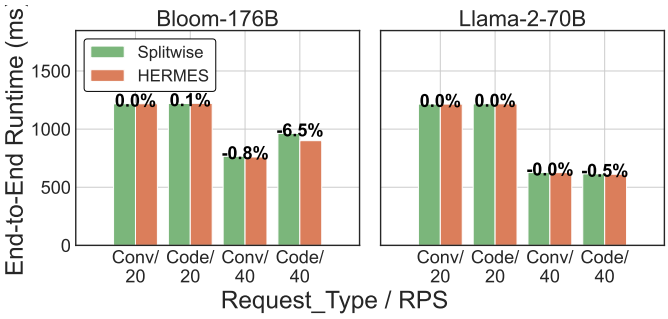


Fig. 5: End-to-end validation results comparing Splitwise and HERMES on an 80-GPU system configured with 8TP.

IV. Understanding impact of stages on inference

In this section we use HERMES to study i) optimal batching strategies for serving reasoning based requests, and ii) hardware affinity requirements for requests requiring retrieval augmented generation(RAG).

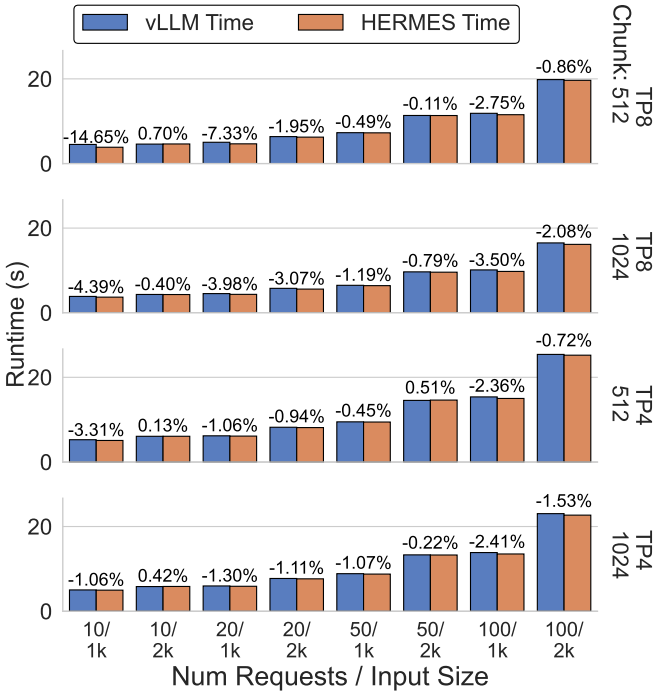


Fig. 6: End-to-end runtime comparation of vLLM and HERMES on different parallelization with HGX:H100x8 running Llama3.1-70B. For each hardware configuration, we vary the context length, number of requests, and chunk size.

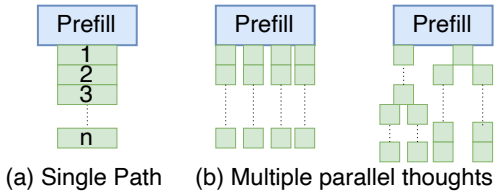


Fig. 7: Scaling test time compute through single-path and multiple parallel reasoning thoughts.

A. Reasoning based models

Ever since release of deepseek-R1 [66], increasing the inference-time compute budget for LLMs—also referred to as *reasoning*—has become a widely adopted approach to generate higher-quality responses. Scaling test-time compute [28], [67], [68], [69] involves providing additional compute resources at inference time, which typically results in generating more output tokens or performing multiple reasoning steps. As shown in Fig. 7, reasoning leads to longer auto-regressively generated thought chains.

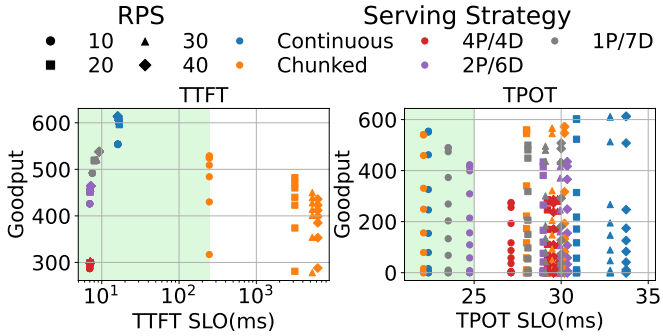
There are broadly two types of reasoning strategies: *single-path reasoning* and *multi-path reasoning*. In *single-path reasoning* [70], [71], [72], [73], [74], [75], the task is broken down into a linear sequence of intermediate steps, where each step builds upon the previous one. In contrast, *multi-path reasoning* [28], [76], [77], [78], [79], [80], [81]

decomposes the task into multiple reasoning branches that are processed in parallel. Each branch requires a separate KV cache, leading to increased memory usage.

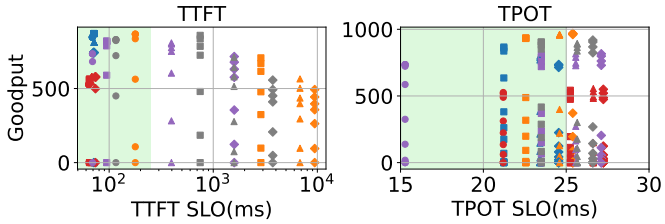
The qualitative impact of reasoning is an increase in the number of generated tokens due to the intermediate reasoning steps. With multi-path reasoning, while the prefill context is shared across branches, the explosion in branches significantly increases KV memory demands.

Reasoning Implementation: To model *single-path reasoning*, we scale the output tokens by approximately 8–32× per request [28]. To model multi-path reasoning, we scale output tokens by 4–16×, while assuming each request spawns 8 parallel thought branches. We simulate a worst-case scenario where all thought branches are independent, maximizing KV cache demand. Prefill KV caches are shared across the branches to avoid redundant memory usage.

Effect of batching strategies with reasoning: While it's intuitive that reasoning-based LLMs increase memory usage—thereby limiting the maximum batch size—there is limited guidance on optimal scheduling configurations for different reasoning-heavy use cases. Using HERMES , we evaluate different scheduling strategies for such workloads. Fig. 8 shows the number of requests that meet the SLOs for TTFT and TPOT. We observe that *chunked prefill* achieves high decode throughput but suffers from high TTFTs, especially at higher



(a) Input:AzureConv, Output:2k/σ = 30% w 8 Parallel Branches.



(b) Input:AzureCode, Output: 2k/σ = 30% w 4 Parallel Branches.

Fig. 8: Comparison goodput(Requests satisfying the SLO) for different batching strategies. Running Llama-3.1-70B on 64 GPU(8xTP8). Green region indicates SLO complaint region.

request arrival rates. This is because long input sequences delay subsequent requests, increasing queueing latency. *Continuous batching* provides strong throughput while respecting TTFT SLOs for conversational workloads, which usually have shorter input sequences. For code generation, continuous batching still optimizes TTFTs, but *disaggregated serving* outperforms overall. It significantly reduces TPOT while keeping TTFTs comparable to continuous batching.

> ✿ *Chunked prefill* offers strong decode throughput, but is suitable only for use cases with relaxed TTFT requirements.
> ✿ For short-context chat workloads: *Continuous batching* provides high throughput while meeting TTFT SLOs.
> ✿ For long-context code generation: *Disaggregated serving*, with more decode clients, is optimal. It delivers significantly lower TPOT latency while maintaining low TTFT latency similar to continuous batching.

B. Retrieval Augmented Generation

Most modern LLM inference pipelines integrate RAG to improve factual accuracy and reduce hallucinations. The RAG pipeline typically introduces three components before the prefill stage: *query embedding*, *context retrieval*, and *re-ranking*. These components can either be co-located on the same system or disaggregated across different clients. Additionally, each component has distinct hardware requirements: the *embedding stage* is compute-intensive (similar to prefill), whereas the *retrieval* stage is memory-bound due to large-scale database lookups.

In this section, we study: i) The impact of *co-locating* all three stages on the same hardware versus *disaggregating* the embedding model and the retrieval + re-ranking module; and ii) The *importance of bandwidth* between the re-ranker client and the prefill client.

This helps us evaluate whether high-bandwidth CPU-GPU links (e.g., Grace Hopper) offer significant benefits, or whether lower-bandwidth links like PCIe are sufficient.

We evaluate three hardware configurations: 1. *Grace-like CPU* for both Embedding and Retrieval, 2. *Sapphire Rapids-like CPU* for Embedding and Retrieval, 3. *A100 GPU* for Embedding + *Grace CPU* for Retrieval. In all setups, *prefill and decode* are executed on a single *H100 GPU* running LLaMA-3.1-8B. The prefill client connects to the retrieval client via *PCIe 4.0 x4 (32GB/s)*.

We evaluate two embedding models: *E5-Base* [31], [30], [29], [82], and *Mistral-7B*. The retrieval uses the *IVF-PQ algorithm* with: 4M centroids, 50 probes, 5K points per probe. After re-ranking, 20 documents (512 tokens each) are selected, adding 10K context tokens. We use queries from the *Azure Conversational dataset*.

Fig. 9 shows that placing large embedding models on smaller CPUs leads to severe performance bottlenecks, significantly increasing TTFT. Offloading the embedding computation to

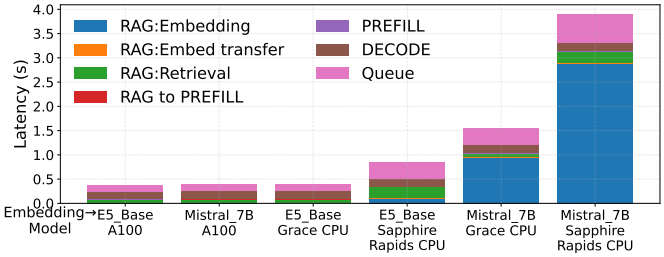


Fig. 9: Understanding the bottleneck of LLM inference pipeline for different embedding models on different HWs.

a faster NPU, like an A100, drastically reduces embedding latency—even for large models like Mistral-7B. We also observe that the time spent transferring retrieved context to the prefill client is negligible (under 1% of total runtime), even on modest PCIe bandwidth.

Thus, using high compute HW for the embedding stage is more critical than maximizing CPU-GPU bandwidth for context transfer.

> For large embedding models (e.g., Mistral-7B), *embedding time* becomes a *major bottleneck*; offloading it to a high-compute NPU significantly improves performance.
> *CPU-GPU bandwidth* is rarely a bottleneck for typical (10K token) context transfers—even with PCIe4.0x4, transfer time accounts for less than 1% of total runtime.

V. Navigating Design Choices with HERMES

HERMES allows us to model different real-world scenarios and as a result make more efficient LLM inference pipelines.

A. Impact of Batching Methods in Online Serving

Online serving workloads vary significantly in input/output characteristics, service level objectives (SLOs), and available hardware resources. As a result, identifying optimal batching strategies is a complex and context-dependent task. Previous works that propose chunked batching [16], [42] typically compare against continuous batching strategies (e.g., vLLM/Orca), while others exploring disaggregated serving [17], [43] primarily benchmark against vLLM. These studies mostly focus on pipelines limited to prefill and decode stages. To date, no work has systematically compared batching strategies across different request compositions.

To the best of our knowledge, this is the first comprehensive study evaluating multiple batching strategies across diverse LLM inference pipelines.

Hardware Setup: We evaluate performance on two configurations: a single HGX platform (8 GPUs) and a full HGX rack (64 GPUs), running various numbers of Llama3-70B clients. Client runtimes are predicted using a polynomial model trained on real hardware traces, as described in Section III-C. We model intra- and inter-platform communication using measured bandwidth and latency from Nvidia HGX systems, based on

Calculon [83]. Power consumption is estimated using power numbers generated from GenZ [45].

SLOs: To determine the maximum throughput each cluster configuration can support, we evaluate P50, P90, and P99 SLOs for both time-to-first-token (TTFT) and time-per-output-token (TPOT). Table II lists the acceptable slowdowns from the baseline TTFT (250 ms, or 1000 ms for RAG/memory retrieval) and TPOT (25 ms). All six SLOs must be satisfied. Slightly looser bounds are used for TTFT since its effect on overall latency is relatively smaller.

SLOs(ms)	P50	P90	P99
TTFT	2x	3x	6x
TPOT	1.25x	1.5x	5x

TABLE II: SLOs slowdown compared to the baseline SLO.

1) Comparing Batching Strategies Across LLM Pipelines: We examine the performance of various batching strategies on workloads drawn from conversation and coding tasks [64], evaluated across three types of LLM pipelines: i) Standard Prefill-Decode, ii) RAG + Prefill-Decode, and iii) KV Cache Retrieval.

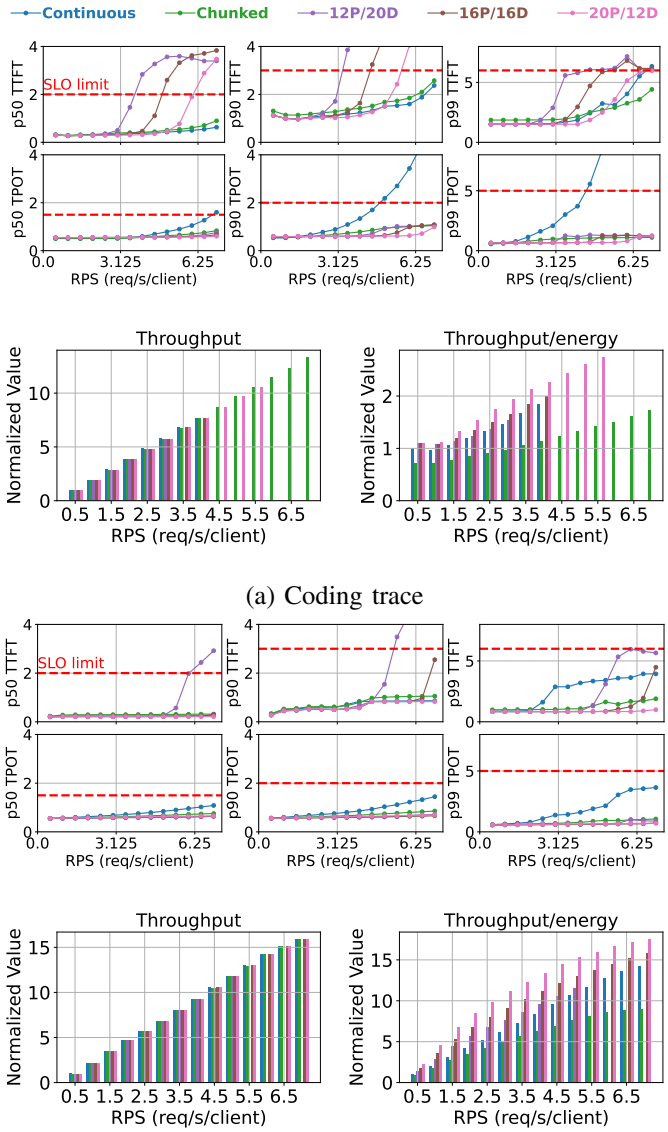
We compare five batching methods, each serving 32 clients: *Continuous* (similar to vLLM), *Chunked* (Sarathi), and *Disaggregated* (Splitwise), where prefill and decode clients are separated.

For each method, we gradually increase the per-client request rate. As the rate rises, TTFT and TPOT degrade. Among the configurations meeting all six SLOs, we report normalized throughput (output tokens/sec) and throughput per unit energy, considering continuous batching with lowest injection rate as the normalization baseline.

Fig. 10(a) presents results for the coding trace, characterized by long inputs and short generations. Here, chunked and disaggregated batching achieve the highest throughput by allowing concurrent prefill and decode operations. Continuous batching favors prefill tokens, yielding low TTFT but poor TPOT under high load. While chunked batching achieves near-peak throughput across all input rates, disaggregated serving (20P/12D) yields higher throughput per energy due to better utilization: decode-only clients, being memory-bound, consume less compute power.

Fig. 10(b) shows results for the conversation trace, which features shorter inputs and outputs. In this case, disaggregated serving (20P/12D) consistently achieves the lowest TTFT and the highest throughput and throughput per energy. Chunked and continuous methods perform comparably in raw throughput but fall short in energy efficiency.

RAG Requests: Including a RAG stage introduces 3K additional retrieval tokens, extending prefill duration. Fig. 11 compares the throughput and throughput per energy across different ingress rates. LLM pipeline with RAG stage is able to sustain comparatively lower injection rate than regular requests due to longer prefill stages. Here, chunked and disaggregated (20P/12D) strategies achieve the highest throughput, with disaggregated again offering superior energy efficiency.



(a) Coding trace

(b) Conversation trace

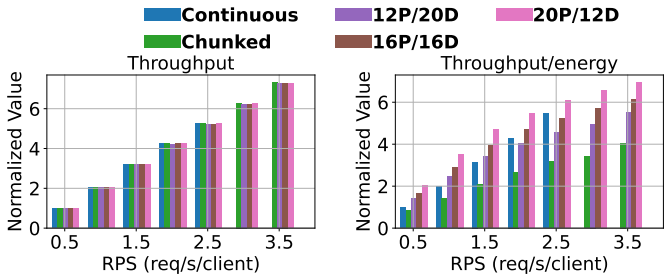
Fig. 10: Comparing different serving strategies for running Llama-3.1-70B on 32 clients of H100 (TP2). Varying P/D legends indicate global disaggregated batching(e.g. 12P/20D indicates 12 prefill and 20 decode clients)

KV Cache Retrieval Requests: For requests that depend on previously cached context (3K tokens), we assume cache availability without recomputation. While retrieval does not extend generation time, it increases input size and thus reduces maximum batch sizes. Fig. 12 compares the throughput and throughput per energy across different ingress rates. Under high input rates, chunked batching offers the best throughput, especially in scenarios with long input contexts like code generation.

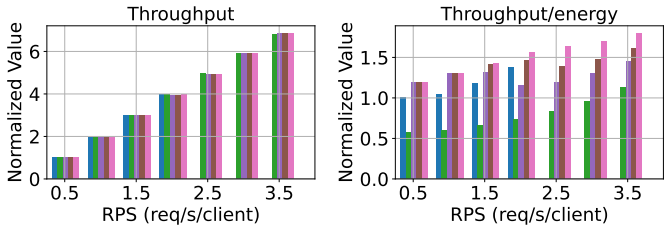
Across all experiments, we simulate diverse request compositions over varied GPU setups, consuming 5,688.88 GPU hours—equivalent to $33,658.67. HERMES was able to simu-

Trace Type	Request Type	System Size	TTFT	Throughput	Throughput/Energy
Code Generation	Regular Prefill-Decode	Small	Continuous	Disaggregated (Medium) Chunked (High)	Disaggregated
		Large	Continuous (Medium) Disaggregated (High)	Disaggregated (Medium) Chunked (High)	Disaggregated
	RAG	Small	Continuous	Chunked	Disaggregated
		Large	Continuous	Chunked/Disaggregated	Disaggregated
	Memory Cache Retrieval	Small	Continuous	Disaggregated	Disaggregated
		Large	Continuous (Medium) Disaggregated (High)	Disaggregated (Medium) Chunked (High)	Disaggregated
Conv (Chatbots)	Regular Prefill-Decode	Small	Disaggregated	Disaggregated	Disaggregated
		Large	Disaggregated	Disaggregated	Disaggregated
	RAG	Small	Continuous	Chunked/Disaggregated	Disaggregated
		Large	Continuous	Chunked/Disaggregated	Disaggregated
	Memory Cache Retrieval	Small	Continuous	Chunked/Disaggregated	Disaggregated
		Large	Disaggregated	Disaggregated	Disaggregated
	Reasoning (Scaling output w parallel thoughts)	Small	Continuous	Continuous	Disaggregated (Low) Continuous (Medium)
		Large	Continuous	Disaggregated (Low) Continuous (Medium)	Disaggregated (Low) Continuous (Medium)

TABLE III: ⚙ Batching Strategy Recommendation based on input trace, inference pipeline, and serving system size. Small serving setup with a single platform (4xTP2) and large serving setup simulating a rack (32xTP2) for serving LLama3-70B. For cases with multiple recommendations, Low/Medium/High refers to per client incoming request rate (e.g. For optimizing throughput in code generation with regular prefill decode, disaggregated batching is recommended at medium request rates,i.e.3-4 req/s, and chunked batch is recommended at high request rate, i.e. 5-6 req/s. Last three columns indicate the optimization objective, minimizing TTFT and maximizing throughput, throughput/energy.

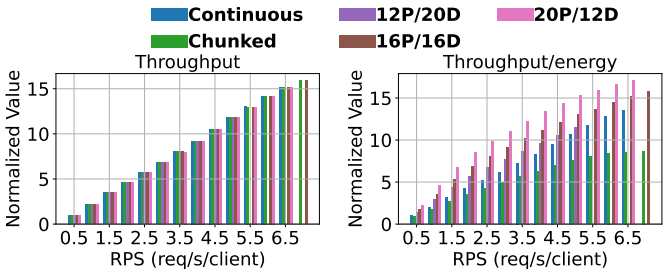


(a) Conversation trace

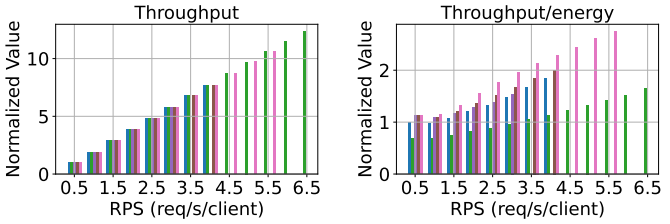


(b) Coding trace

Fig. 11: Comparing different serving strategies when with RAG based pipeline for running Llama-3.1-70B on 32 clients of H100 (TP2).



(a) Conversation trace



(b) Coding trace

Fig. 12: Comparing different serving strategies when with memory retrieval based pipeline for running Llama-3.1-70B on 32 clients of H100 (TP2).

late with 16 core M1 CPUs in 8 hours. Table III summarizes our recommended batching strategies for each request type.

2) Scaling Clients: To evaluate scalability, we increase the number of clients from 2 to 32. For each client count and batching strategy, we determine the highest per-client request rate that meets all SLOs, using Azure conversational traces. Each client is powered by 2×H100 GPUs for Llama3-70B inference.

Fig. 13 shows the effective goodput as we vary the generation SLAs. Chunked batching sustains higher input rates under relaxed SLOs. However, as SLO constraints tighten, its performance drops significantly. Disaggregated batching with a 60% prefill ratio maintains SLO compliance even at higher

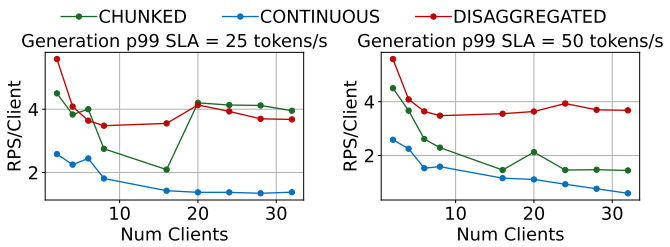


Fig. 13: Effective goodput meeting generation service level agreement(99% of requests should meet the token generation target). We compare different serving strategies when scaling number of serving clients(Llama 3 70B/H100 using tensor parallelism across 2 GPUs).

input rates, making it the most robust under strict latency requirements.

B. Remote KV Cache Storage.

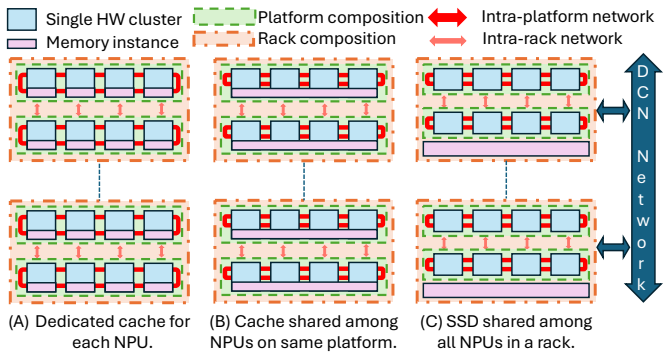


Fig. 14: Various different cache storage solution.

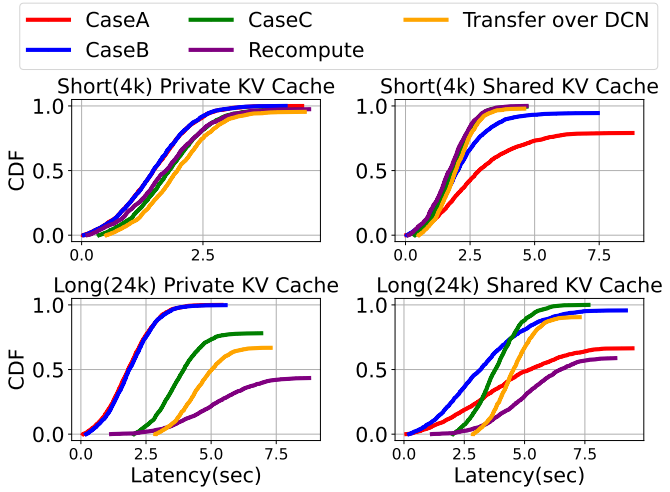


Fig. 15: Comparing different platform architectures for storing past caches storage. Serving 128 clients of Llama-3.1-70B using 4 HGX Racks(Each with 64 H100-like NPUs).

Efficient memory cache retrieval is critical for AI inference engines, where rapid access to context directly impacts overall system performance. In this study, we analyze key trade-offs in cache storage granularity, strategies for cache recomputation, and conditions under which transferring data over a Data Center Network (DCN) is warranted.

Target Usecase: The analysis addresses two principal scenarios: *i) private key-value (KV) caches*—designed for individual user contexts(eg. Personal AI chat engines like ChatGPT, Deepseek). These private KV cache can be accessed by future queries from the same user. *ii) Shared KV caches*—often used in multi-user settings (e.g., enterprise AI or shared codebases) to enable multiple users to access a large corpus ($O(10^{10})$ tokens) of documents or code. Generally these caches have some KV caches hotspots that would be accessed at much higher frequency then rest of the KV cache.

Hardware design space: Typical AI serving architecture with storage memory can be comprised into various different cache storage solutions. We classify these into three categories (Fig. 14): (A) a dedicated cache per client, (B) a platform-level shared cache with shared access by 2-8 clients, and (C) a rack-level shared cache with shared access by 32-64 clients Each configuration presents distinct trade-offs in terms of capacity, bandwidth, and latency.

Experimental Setup: We evaluate these architecture choices on a high-performance cluster comprising 256 GPUs, organized into 128×H100:TP2 nodes distributed across 4 racks, interconnected via NVLink and PCIe. The experiments simulate two distinct workloads: short KV cache retrieval (4K tokens) and long KV cache retrieval (24K tokens), spanning both private (user-specific) and shared (enterprise/global) contexts. For both workload the requests are sampled from AzureConv and injected to the system at rate of 240 request per sec with Poisson distribution. Three cache storage tiers are examined (Fig. 14): (A) a dedicated per-client LPDDR based cache offering 1 TB capacity at 128 GB/s bandwidth; (B) a platform-shared cache with 4 TB capacity at 32 GB/s accessed by 4 clients; (C) a rack-shared cache with 32 TB capacity at 2 GB/s accessed by 32 clients. Additional also study the configuration a rack-level cache(Similar to C) augmented with data center network for inter-rack cache transfers. We assume inter-rack connectivity to have 128 GB/s Ethernet links. The final scenario is where KV cache is unavailable and the KV cache for past contexts needs to be recomputed. End-to-end request serving latency serves as the primary evaluation metric, providing a holistic measure of system performance under varied cache configurations and workload conditions.

Fig. 15 shows the end-2-end latency distribution cdf for the design space. For private KV caches, we find that platform-level shared cache (config B) offers best request latency T90. Conversely, for shared global KV caches, a rack-level shared cache (config C) is superior, as it delivers higher aggregate capacity and maintains acceptable performance despite a modest reduction in per-client bandwidth.

For short KV caches (~ 4K tokens), the overhead associated with recomputation is low, making it a competitive alternative to

direct cache retrieval, particularly when it avoids the additional delay introduced by DCN transfers. However, as the KV cache size increases (24K tokens), the recomputation overhead becomes prohibitive; in such cases, utilizing a rack-level cache to directly retrieve stored data proves to be more efficient. Although DCN transfers (config C + DCN) can serve as a fallback mechanism in instances of replica overload, the inherent link latency (approximately 20 msec) renders this approach less attractive for large caches.

- Platform-level shared cache (B) is best suited for private KV caches as it balances speed and retrieval speed.
- Rack-level shared cache (C) is optimal for shared global KV caches: Provides low-latency access and efficient inter-node sharing.
- Recomputation is a viable strategy for short KV caches especially when cache reuse is limited.

VI. Related Work

Prior works leverage the predictability of DNN training iterations [84], [85], [86], [87], [88] to model training performance. Several simulation frameworks have been proposed to model LLM systems. LLMCompass[44] and GenZ[45] provide detailed modeling capabilities for single-client configurations.

Vidur [14] supports multi-client simulation but assumes client homogeneity and is restricted to modeling existing system configurations. It lacks support for heterogeneous client setups, disaggregated hardware for prefill and decode stages, or speculative scenarios involving future system architectures. Similarly, LLMServingSim [13] does not support chunked prefill or disaggregated batching, which are increasingly common in production-grade LLM deployments. Splitwise-sim [15] models three pools for hardware clients representing prefill, decode and mixed pool. Similar to LLMServingsim it doesn't model chunked batching. Consequently, both Vidur, LLMServingSim and Splitwise-sim fall short in modeling advanced, multi-stage LLM inference pipelines. In contrast, HERMES is the first simulator designed to support end-to-end modeling of real-world LLM inference pipelines across heterogeneous HW configurations.

VII. Conclusion

Modern LLM inference pipelines demand simulators that can model complex, heterogeneous workflows—something existing tools fail to provide. We present HERMES a high-fidelity, event-driven simulation framework that captures the full spectrum of inference stages across diverse hardware setups. HERMES supports flexible batching, multi-model execution, and detailed memory modeling, enabling accurate evaluation of architectural trade-offs. Through case studies, we show how HERMES offers actionable insights into KV cache design, batching strategies, and hardware-software co-design. Looking ahead, HERMES can be extended for exploring optimal configuration of future chips, developing new adaptive schedulers and simulating future multi-agent LLM deployments.

VIII. Acknowledgment

This work was supported in part by CoCoSys, one of seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA.

References

[1] N. Brandes, D. Ofer, Y. Peleg, N. Rappoport, and M. Linial, "Proteinbert: a universal deep-learning model of protein sequence and function," *Bioinformatics*, vol. 38, pp. 2102–2110, 4 2022. [Online]. Available: https://github.com/nadavbra/protein_bert

[2] HyunSeobKim, "Chem-bert: Molecular representation learning." [Online]. Available: https://github.com/HyunSeobKim/CHEM-BERT

[3] D. Kondratyuk, L. Yu, X. Gu, J. Lezama, J. Huang, G. Schindler, R. Hornung, V. Birodkar, J. Yan, M.-C. Chiu, K. Somandepalli, H. Akbari, Y. Alon, Y. Cheng, J. Dillon, A. Gupta, M. Hahn, A. Hauth, D. Hendon, A. Martinez, D. Minnen, M. Sirotenko, K. Sohn, X. Yang, H. Adam, M.-H. Yang, I. Essa, H. Wang, D. A. Ross, B. Seybold, and L. Jiang, "Videopoet: A large language model for zero-shot video generation," 2024.

[4] OpenAI, "Chatgpt." [Online]. Available: https://openai.com/chatgpt

[5] Google, "Introducing gemini: Google's most capable ai model yet," 2023. [Online]. Available: https://blog.google/technology/ai/google-gemini-ai/

[6] MIcrosoft, "Github copilot · your ai pair programmer." [Online]. Available: https://github.com/features/copilot

[7] E. Roivainen, "I gave chatgpt an iq test. here's what i discovered — scientific american." [Online]. Available: https://www.scientificamerican.com/article/i-gave-chatgpt-an-iq-test-heres-what-i-discovered/

[8] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," 2020.

[9] I. is beautiful, "The rise and rise of a.i. large language models (llms)." [Online]. Available: https://informationisbeautiful.net/visualizations/the-rise-of-generative-ai-large-language-models-llms-like-chatgpt/

[10] NVIDIA, "Nvidia h100 tensor core gpu architecture," 2022, https://resources.nvidia.com/en-us-tensor-core.

[11] "Intel hls-gaudi2." [Online]. Available: https://habana.ai/wp-content/uploads/2023/10/HLS-Gaudi2%5FDatasheet%5F10%5F23.pdf

[12] "instinct-mi325x-datasheet.pdf," https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/product-briefs/instinct-mi325x-datasheet.pdf, 2024, (Accessed on 11/22/2024).

[13] J. Cho, M. Kim, H. Choi, G. Heo, and J. Park, "Llmservingsim: A hw/sw co-simulation infrastructure for llm inference serving at scale," 2024. [Online]. Available: https://arxiv.org/abs/2408.05499

[14] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. Gulavani, R. Ramjee, and A. Tumanov, "Vidur: A large-scale simulation framework for llm inference," 2024. [Online]. Available: https://arxiv.org/abs/2405.05465

[15] Mutinifni, "SplitwiseSim: LLM Serving Cluster Simulator," https://github.com/Mutinifni/splitwise-sim, 2024, accessed: 2025-04-10.

[16] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee, "Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills," *arXiv preprint arXiv:2308.16369*, 2023.

[17] P. Patel, E. Choukse, C. Zhang, Íñigo Goiri, A. Shah, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative llm inference using phase splitting," 2023.

[18] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {Transformer-Based} generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.

[19] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[20] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," 2017. [Online]. Available: https://arxiv.org/abs/1702.08734

[21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," 2021. [Online]. Available: https://arxiv.org/abs/2005.11401

[22] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," 2020. [Online]. Available: https://arxiv.org/abs/2004.04906

[23] J. Mazanec and O. Hamzaoui, "Choose the k-nn algorithm for your billion-scale use case with opensearch — aws big data blog," 9 2022, [Online; accessed 2025-03-08]. [Online]. Available: https://aws.amazon.com/blogs/big-data/choose-the-k-nn-algorithm-for-your-billion-scale-use-case-with-opensearch/

[24] A. Chirkin, "Accelerating vector search: Nvidia cuvs ivf-pq part 1, deep dive — nvidia technical blog," 7 2024, [Online; accessed 2025-03-08]. [Online]. Available: https://developer.nvidia.com/blog/accelerating-vector-search-nvidia-cuvs-ivf-pq-deep-dive-part-1/

[25] "Choose the k-nn algorithm for your billion-scale use case with opensearch — aws big data blog," 9 2022, [Online; accessed 2025-04-11]. [Online]. Available: https://aws.amazon.com/blogs/big-data/choose-the-k-nn-algorithm-for-your-billion-scale-use-case-with-opensearch/

[26] K. Jain, A. Parayil, A. Mallick, E. Choukse, X. Qin, J. Zhang, Íñigo Goiri, R. Wang, C. Bansal, V. Rühle, A. Kulkarni, S. Kofsky, and S. Rajmohan, "Intelligent router for llm workloads: Improving performance through workload-aware load balancing," 2025. [Online]. Available: https://arxiv.org/abs/2408.13510

[27] S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "ASTRA-SIM: Enabling sw/hw co-design exploration for distributed dl training platforms," in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2020.

[28] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling llm test-time compute optimally can be more effective than scaling model parameters," 2024. [Online]. Available: https://arxiv.org/abs/2408.03314

[29] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, "Text embeddings by weakly-supervised contrastive pre-training," *arXiv preprint arXiv:2212.03533*, 2022.

[30] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Improving text embeddings with large language models," *arXiv preprint arXiv:2401.00368*, 2023.

[31] ——, "Multilingual e5 text embeddings: A technical report," 2024. [Online]. Available: https://arxiv.org/abs/2402.05672

[32] A. Chirkin, "Accelerating vector search: Nvidia cuvs ivf-pq part 1, deep dive — nvidia technical blog," 7 2024, [Online; accessed 2025-04-10]. [Online]. Available: https://developer.nvidia.com/blog/accelerating-vector-search-nvidia-cuvs-ivf-pq-deep-dive-part-1/

[33] W. Jiang, M. Zeller, R. Waleffe, T. Hoefler, and G. Alonso, "Chameleon: a heterogeneous and disaggregated accelerator system for retrieval-augmented language models," 2025. [Online]. Available: https://arxiv.org/abs/2310.09949

[34] W. Jiang, S. Subramanian, C. Graves, G. Alonso, A. Yazdanbakhsh, and V. Dadu, "Rago: Systematic performance optimization for retrieval-augmented generation serving," 2025. [Online]. Available: https://arxiv.org/abs/2503.14649

[35] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, "Cacheblend: Fast large language model serving with cached knowledge fusion," *arXiv preprint arXiv:2405.16444*, 2024.

[36] Y. Cheng, K. Du, J. Yao, and J. Jiang, "Do large language models need a content delivery network?" *arXiv preprint arXiv:2409.13761*, 2024.

[37] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan *et al.*, "Cachegen: Kv cache compression and streaming for fast large language model serving," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 38–56.

[38] "Automatic prefix caching — vllm," [Online; accessed 2025-04-11]. [Online]. Available: https://docs.vllm.ai/en/latest/design/v1/prefix_caching.html

[39] NVIDIA, "Github - nvidia/fastertransformer: Transformer related optimization, including bert, gpt," [Online; accessed 2025-04-10]. [Online]. Available: https://github.com/NVIDIA/FasterTransformer

[40] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 611–626. [Online]. Available: https://doi.org/10.1145/3600006.3613165

[41] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," *Proceedings of 18th USENIX Symposium on Operating Systems Design and Implementation, 2024, Santa Clara*, 2024.

[42] C. Holmes, M. Tanaka, M. Wyatt, A. A. Awan, J. Rasley, S. Rajbhandari, R. Y. Aminabadi, H. Qin, A. Bakhtiari, L. Kurilenko *et al.*, "Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference," *arXiv preprint arXiv:2401.08671*, 2024.

[43] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving," 2024.

[44] H. Zhang, A. Ning, R. Prabhakar, and D. Wentzlaff, "A hardware evaluation framework for large language model inference," 2023. [Online]. Available: https://arxiv.org/abs/2312.03134

[45] A. Bambhaniya, R. Raj, G. Jeong, S. Kundu, S. Srinivasan, M. Elavazhagan, M. Kumar, and T. Krishna, "Demystifying platform requirements for diverse llm inference use cases," *arXiv preprint arXiv:2406.01698*, 2024.

[46] C. to Wikimedia projects, "Rubin (microarchitecture) - wikipedia," 6 2024, [Online; accessed 2025-04-11]. [Online]. Available: https://en.wikipedia.org/wiki/Rubin_(microarchitecture)

[47] A. Vahdat, "Ironwood: The first google tpu for the age of inference," 4 2025, [Online; accessed 2025-04-11]. [Online]. Available: https://blog.google/products/google-cloud/ironwood-tpu-age-of-inference/

[48] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to dnn accelerator evaluation," in *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019, pp. 304–315.

[49] H. Kwon, P. Chatarasi, V. Sarkar, T. Krishna, M. Pellauer, and A. Parashar, "Maestro: A data-centric approach to understand reuse, performance, and hardware cost of dnn mappings," *IEEE Micro*, vol. 40, no. 3, pp. 20–29, 2020.

[50] S.-C. Kao, S. Subramanian, A. Bambhaniya, and T. Krishna, "FRAME: Fast Roofline Analytical Modeling and Estimation," 2022. [Online]. Available: https://github.com/maestro-project/frame

[51] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," in *Low-Power Computer Vision*. Chapman and Hall/CRC, 2022, pp. 291–326.

[52] H. Kang, Q. Zhang, S. Kundu, G. Jeong, Z. Liu, T. Krishna, and T. Zhao, "Gear: An efficient kv cache compression recipe for near-lossless generative inference of llm," 2024.

[53] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," PMLR, pp. 38 087–38 099, 2023.

[54] E. Frantar and D. Alistarh, "Sparsegpt: Massive language models can be accurately pruned in one-shot," in *International Conference on Machine Learning*. PMLR, 2023, pp. 10 323–10 337.

[55] A. R. Bambhaniya, A. Yazdanbakhsh, S. Subramanian, S.-C. Kao, S. Agrawal, U. Evci, and T. Krishna, "Progressive gradient flow for robust n:m sparsity training in transformers," 2024.

[56] G. Jeong, P.-A. Tsai, A. R. Bambhaniya, S. W. Keckler, and T. Krishna, "Abstracting sparse dnn acceleration via structured sparse tensor decomposition," 2024.

[57] A. R. Bambhaniya, A. Yazdanbakhsh, S. Subramanian, and T. Krishna, "Accelerating attention based models via HW-SW co-design using fine-grained sparsification," in *Architecture and System Support for Transformer Models (ASSYST @ISCA 2023)*, 2023. [Online]. Available: https://openreview.net/forum?id=xd5qPRXLl7

[58] S. Kim, K. Mangalam, S. Moon, J. Malik, M. W. Mahoney, A. Gholami, and K. Keutzer, "Speculative decoding with big little decoder," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[59] Y. Fu, P. Bailis, I. Stoica, and H. Zhang, "Break the sequential dependency of llm inference using lookahead decoding," *arXiv preprint arXiv:2402.02057*, 2024.

[60] M. Elbayad, J. Gu, E. Grave, and M. Auli, "Depth-adaptive transformer," *arXiv preprint arXiv:1910.10073*, 2019.

[61] Y. Chen, X. Pan, Y. Li, B. Ding, and J. Zhou, "Ee-llm: Large-scale training and inference of early-exit large language models with 3d parallelism," *arXiv preprint arXiv:2312.04916*, 2023.

[62] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," 2022.

[63] S.-C. Kao, S. Subramanian, G. Agrawal, and T. Krishna, "An optimized dataflow for mitigating attention performance bottlenecks," *arXiv preprint arXiv:2107.06419*, 2021.

[64] M. Azure, "Azure Public Dataset: Azure LLM Inference Trace 2023," https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md, 2023, accessed: 2025-04-10.

[65] vLLM contributors, "Add Splitwise Implementation to vLLM," https://github.com/vllm-project/vllm/pull/2809, 2024, accessed: 2025-04-10.

[66] DeepSeek-AI, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," 2025. [Online]. Available: https://arxiv.org/abs/2501.12948

[67] N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, and T. Hashimoto, "s1: Simple test-time scaling," 2025. [Online]. Available: https://arxiv.org/abs/2501.19393

[68] W. Yang, S. Ma, Y. Lin, and F. Wei, "Towards thinking-optimal scaling of test-time compute for llm reasoning," 2025. [Online]. Available: https://arxiv.org/abs/2502.18080

[69] Y. Chen, X. Pan, Y. Li, B. Ding, and J. Zhou, "Simple and provable scaling laws for the test-time compute of large language models," 2025. [Online]. Available: https://arxiv.org/abs/2411.19477

[70] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models." [Online]. Available: http://arxiv.org/abs/2201.11903

[71] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners." [Online]. Available: http://arxiv.org/abs/2205.11916

[72] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in hugging face." [Online]. Available: http://arxiv.org/abs/2303.17580

[73] B. Y. Lin, Y. Fu, K. Yang, F. Brahman, S. Huang, C. Bhagavatula, P. Ammanabrolu, Y. Choi, and X. Ren, "SwiftSage: A generative agent with fast and slow thinking for complex interactive tasks." [Online]. Available: http://arxiv.org/abs/2305.17390

[74] S. S. Raman, V. Cohen, E. Rosen, I. Idrees, D. Paulius, and S. Tellex, "Planning with large language models via corrective re-prompting." [Online]. Available: https://openreview.net/forum?id=cMDMRBe1TKs

[75] B. Xu, Z. Peng, B. Lei, S. Mukherjee, Y. Liu, and D. Xu, "Rewoo: Decoupling reasoning from observations for efficient augmented language models," 2023. [Online]. Available: https://arxiv.org/abs/2305.18323

[76] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models." [Online]. Available: http://arxiv.org/abs/2203.11171

[77] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models." [Online]. Available: http://arxiv.org/abs/2305.10601

[78] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents." [Online]. Available: http://arxiv.org/abs/2201.07207

[79] B. Sel, A. Al-Tawaha, V. Khattar, R. Jia, and M. Jin, "Algorithm of thoughts: Enhancing exploration of ideas in large language models." [Online]. Available: http://arxiv.org/abs/2308.10379

[80] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler, "Graph of thoughts: Solving elaborate problems with large language models," vol. 38, no. 16, pp. 17 682–17 690. [Online]. Available: http://arxiv.org/abs/2308.09687

[81] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, and Z. Hu, "Reasoning with language model is planning with world model." [Online]. Available: http://arxiv.org/abs/2305.14992

[82] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "Mteb: Massive text embedding benchmark," *arXiv preprint arXiv:2210.07316*, 2022. [Online]. Available: https://arxiv.org/abs/2210.07316

[83] M. Isaev, N. Mcdonald, L. Dennison, and R. Vuduc, "Calculon: a methodology and tool for high-level co-design of systems and large language models," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3581784.3607102

[84] H. Zhu, A. Phanishayee, and G. Pekhimenko, "Daydream: Accurately estimating the efficacy of optimizations for DNN training," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, Jul. 2020, pp. 337–352. [Online]. Available: https://www.usenix.org/conference/atc20/presentation/zhu-hongyu

[85] W. Xiao, R. Bhardwaj, R. Ramjee, M. Sivathanu, N. Kwatra, Z. Han, P. Patel, X. Peng, H. Zhao, Q. Zhang, F. Yang, and L. Zhou, "Gandiva: Introspective cluster scheduling for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. Carlsbad, CA: USENIX Association, Oct. 2018, pp. 595–610. [Online]. Available: https://www.usenix.org/conference/osdi18/presentation/xiao

[86] M. Sivathanu, T. Chugh, S. S. Singapuram, and L. Zhou, "Astra: Exploiting predictability to optimize deep learning," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 909–923. [Online]. Available: https://doi.org/10.1145/3297858.3304072

[87] G. X. Yu, Y. Gao, P. Golikov, and G. Pekhimenko, "A runtime-based computational performance predictor for deep neural network training," 2021. [Online]. Available: https://arxiv.org/abs/2102.00527

[88] D. K. Kadiyala, S. Rashidi, T. Heo, A. R. Bambhaniya, T. Krishna, and A. Daglis, "Comet: A comprehensive cluster design methodology for distributed deep learning training," 2024. [Online]. Available: https://arxiv.org/abs/2211.16648